# Python Engineering - Reference Guide

## Class 08 - Data Processing with NumPy - Expanded version

Reference guide for boolean masks, filtering, statistics, and data processing. For each function this version explains what it does, what it receives, what it returns, and gives a small example.

## 1. Data-processing mental pattern

```
ENGINEERING QUESTION
        -> CONDITION
           -> BOOLEAN MASK
              -> FILTER / COUNT / INDICES
                 -> STATISTIC / RESULT
```

## 2. What is a boolean mask?

A boolean mask is an array of True/False values. Each mask position corresponds to one position in the original data.

```
heights = np.array([20, 80, 120, 150])

mask = heights > 100

# [False False True True]
```

## 3. What does A[mask] mean?

`A[mask]` is not a function. It is **boolean indexing**. It selects only those values or rows from A whose corresponding mask entry is True.

```
A = np.array([
    [10, 1],
    [20, 2],
    [30, 3]
])

mask = np.array([True, False, True])

selected = A[mask]

# [[10, 1],
#  [30, 3]]
```

If A is 2D and the mask has one value per row, NumPy uses the mask to select complete rows.

## 4. Combining conditions

| Operation | NumPy syntax | Example | Returns |
|---|---|---|---|
| AND | & | (h > 50) & (v > 0) | True where both conditions are True |
| OR | \| | (h > 100) \| (v < -40) | True where at least one condition is True |
| NOT | ~ | ~(v > 0) | inverts True/False |

With NumPy arrays, conditions should usually be inside parentheses. Use &, | and ~ for element-wise boolean logic.

## 5. np.sum(mask)

**What it does:** adds values. In a boolean mask, True behaves like 1 and False like 0.

**What it receives:** an array or NumPy expression; here, a boolean mask.

**What it returns:** a scalar sum. For a mask, this equals the number of True values.

```
mask = np.array([True, False, True, True])
count = np.sum(mask)
# 3
```

## 6. np.where()

**What it does:** finds positions where a condition is satisfied.

**What it receives:** usually a condition or boolean mask.

**What it returns:** a tuple of index arrays, one per dimension. For 1D data we commonly use `np.where(mask)[0]`.

```
heights = np.array([20, 80, 120, 150])
mask = heights > 100

indices = np.where(mask)[0]
# [2 3]
```

`heights[mask]` returns selected values; `np.where(mask)[0]` returns their positions.

## 7. np.mean()

**What it does:** computes the arithmetic mean.

**What it receives:** a numerical array; optionally an axis.

**What it returns:** a scalar or array depending on whether an axis is used.

```
x = np.array([10.0, 20.0, 30.0])
mean_x = np.mean(x)
# 20.0
```

## 8. np.std()

**What it does:** computes standard deviation, a measure of spread around the mean.

**What it receives:** a numerical array; optionally an axis.

**What it returns:** a scalar or array of standard-deviation values.

```
x = np.array([10.0, 20.0, 30.0])
std_x = np.std(x)
```

## 9. np.min() and np.max()

**What they do:** find minimum or maximum values.

**What they receive:** a numerical array; optionally an axis.

**What they return:** the global min/max or one value per axis.

```
np.min(heights)
np.max(heights)

np.max(matrix, axis=0)
# maximum per column
```

## 10. np.median()

**What it does:** computes the median, the central value after sorting.

**What it receives:** a numerical array.

**What it returns:** a scalar or an array if axis is used.

```
x = np.array([1, 2, 100])
np.median(x)
# 2.0
```

## 11. np.diff()

**What it does:** computes consecutive differences: x[i+1] - x[i].

**What it receives:** an ordered numerical array.

**What it returns:** an array with one fewer element than the input.

```
times = np.array([0.0, 0.1, 0.2, 0.3])
dt = np.diff(times)
# [0.1 0.1 0.1]
```

## 12. np.column_stack()

**What it does:** places several 1D vectors as columns of a 2D matrix.

**What it receives:** a tuple/list of arrays with the same length.

**What it returns:** a 2D matrix with one column per input array.

```
processed = np.column_stack((
    heights,
    velocities,
    times,
    energy
))

# shape -> (N, 4)
```

## 13. np.allclose()

**What it does:** checks whether two arrays or values are approximately equal within tolerance.

**What it receives:** two comparable arrays/values; optional rtol and atol tolerances.

**What it returns:** one boolean: True if all elements are sufficiently close, otherwise False.

```
dt_samples = np.diff(times)

is_constant = np.allclose(
    dt_samples,
    config['dt']
)

# True or False
```

## 14. Filtering by range

```
mask = (
    (times >= 2.0)
    & (times <= 5.0)
)

window = trajectory_array[mask]
```

Input: boolean conditions over the data. Output: a mask that can be used to select only rows inside the range.

## 15. Expanded summary

| Syntax | Receives | Returns | Use |
|---|---|---|---|
| x[mask] | array + boolean mask | selected values | filter elements |
| A[mask] | matrix + row mask | selected rows | filter states |
| np.sum(mask) | boolean mask | number | count True |
| np.where(mask)[0] | boolean mask | indices | find positions |
| np.mean(x) | numeric array | mean | average value |
| np.std(x) | numeric array | std deviation | spread |

| Syntax | Receives | Returns | Use |
|---|---|---|---|
| np.diff(x) | ordered array | N-1 differences | consecutive changes |
| np.column_stack(...) | equal-length vectors | 2D matrix | add variables |
| np.allclose(a,b) | two arrays/values | True/False | compare floats |